COMPLETE SOURCE CODE ARCHITECTURE GUIDE

MANEKIN

FASHION & AI VIRTUAL TRY-ON PLATFORM

MANEKIN FASHION
全体コード完全解説ガイド

〜 初心者でもコードと仕組みが迷わず読める「専門用語かみ砕き」最新版 〜

本ガイドは、オンラインファッションストア「MANEKIN FASHION」およびスマホ・PC上で洋服の重ね着を試せるバーチャル試着室「MANEKIN AI STUDIO」を動かしている **全15個のソースファイル・全31個の通信窓口（ルート）・MySQL 全11個の保存表（テーブル）**の仕組みを、プログラミング初心者にも全体像と処理の流れがかみ砕いて理解できるようまとめた解説書です。 実際のコードの4割以上（10,169行）には「この行は何をしているか」を説明する日本語コメントも入っているため、本書と見比べながら読み進められます。

15	24,403	10,169	11 / 31
構成ファイル総数	総コード行数（実測）	解説コメント (41.7%)	DBの表 / Flaskの窓口

このサイトを支える「6つの工夫」（初心者向け要点）

1. 1ページ完結ストア（SPA）：全24商品・男女切替・カートから注文完了まで、別のWebページを開き直さず1枚のHTMLの中で高速に中身を切り替えます。

2. ポイント＆自動クローゼット連動：初回 1,000pt（1pt＝1円引き）・購入額の1%還元。服を買うと同時に、その人の試着室へ「購入済みアイテム」が自動追加されます。

3. 封印付き会員証（Cookie）＆公式台帳一本化：プログラムから盗み見られない会員証（`manekin_session`・7日有効）で本人確認し、ブラウザ側の古い一時メモは消去して MySQL 台帳だけに統一しました。

4. AI切り抜き＆「服以外」お断り機能：撮った服写真から背景を透明にし、さらに縦10分割のシルエット計算＋画像認識AIで風景やペットの誤登録を防ぎます。

5. 5枚重ねの試着マネキン：靴→パンツ→トップス→上着→帽子の順に画像を重ね、マウスで掴んで（D&D）着せ替えたり、XS〜XLや拡大率を微調整できます。

6. 通信の「追い越し」防止＆店員側再計算：ネット通信の遅延で古い結果が新しい結果を上書きしないよう整理番号（`_MutationSeq` 等）で守り、請求金額はサーバー店員側で必ず計算し直します。

対象フォルダ: MANEKIN2026-2 / データベース名: manekin_db (MySQL)
作成: MANEKIN CODE GUIDE（現コード 15ファイル静的精査済み / HEAD 3eb78a6）

目次 & まずは知っておきたい基本イメージ

第1章 **全体の仕組み・3つの役割分担・全15ファイル一覧**
画面担当（ブラウザ）・受付係（Python Flask）・倉庫の台帳（MySQL）がどう協力して動いているか、全15ファイルの役割と行数を整理します。

第2章 **1枚のHTMLで画面を切り替える魔法（SPA）・デザイン担当（CSS）・商品データ**
ページ再読込なしで画面を切り替える「しおり機能（ハッシュルーティング）」、見た目担当CSSの5分割構成、24商品の `AppState` メモ帳を解説します。

第3章 **お買い物計算・ポイント割引・失敗したら全部やり直す安全なレジ処理**
初期1,000pt・送料ルール・1%還元の計算と、「注文登録・在庫連動・ポイント更新・カート空化」を一気に安全完了させる取引（トランザクション）の流れを追います。

第4章 **別アカウントの履歴が絶対に混ざらない秘密（Cookie会員証 × 整理番号ガード）**
ブラウザの保存箱を撤去（ `purgeAllManekinLocalStorage` ）し、7日間有効の会員証と整理番号（ `_MutationSeq` ）、0.4秒まとめ送信の仕組みを解説します。

第5章 **MANEKIN AI STUDIO（5枚重ねマネキン・AI背景切り抜き・服判定AI）**
画像を5層に重ねる `z-index` 、ドラッグ＆ドロップ着せ替え、サイズ微調整、ブラウザ内での背景切り抜きAI、服以外を弾く2段階AI検査をかみ砕いて説明します。

第6章 **データベース（MySQL manekin_db）全11個の「保存表」完全ガイド**
会員登録・セッション・注文・カート・試着状態など全11個の表どうしのつながり（ER図）や、データ型・自動連動ルール（CASCADE等）を図解します。

第7章 **裏方サーバー（app.py）の全31個の「受付窓口（ルート）」完全ガイド**
ログイン確認（ `/api/auth/me` ）・ログアウト・注文・AI提案・試着保存など、フロント側と会話する全31窓口を役割別・行番号付きで一覧化します。

第8章 **起動＆お掃除スクリプト4種 & 初心者向けキーワード言い換え辞書**
`start_server.bat` や `clear_orders.py` 等の操作手順と、コード内で頻出する横文字をすべて日常イメージに置き換えた辞書を載せています。

これだけ知れば迷わない！「実店舗」でイメージする5つの基本ワード

- **ブラウザ (HTML/CSS/JS)** ＝ **お客さんが見ているお店のフロア・試着室**（ボタンや服の写真が並んでいる画面）。
- **Pythonサーバー (Flask / app.py)** ＝ **カウンター奥の店員さん**（画面からの注文を受け取り、計算・安全チェックをして倉庫へ伝える）。
- **MySQL (manekin_db)** ＝ **奥の倉庫にある公式台帳（Excelの巨大な表の集まり）**（消えないよう会員や注文を永続保管）。
- **API（エーピーアイ）** ＝ **フロアと受付をつなぐ「ご注文窓口」**（例： `/api/orders` は注文専用窓口）。
- **HttpOnly Cookie** ＝ **会員に配る「偽造＆盗み見防止シール付きの会員証」**（JSプログラムからも中身を盗めない安全なカード）。

ソースコード（全15ファイル）の日本語コメントの読み方

各ファイルには `// ...` （JavaScript）、 `/* ... */` （CSS）、 `<!-- ... -->` （HTML）、 `# ...` （Python）、 `REM / ::` （Windowsバッチ）の形で **合計10,169行（全体の 41.7%）** の日本語メモが書かれています。本書に書かれている行番号（例： `L111` ＝111行目）は現行コード（HEAD `3eb78a6` ）の実測値ですので、そのままエディタで行番号ジャンプして読み解くことができます。

第1章 全体の仕組み・3つの役割分担・全15ファイル一覧

MANEKIN FASHION は、お客さんが触る**「ブラウザ（見た目担当）」**、裏方で計算やチェックを行う**「Python サーバー（店員担当）」**、データを安全に長期保管する**「MySQL（倉庫の台帳担当）」**の3つが、手紙（JSON や SQL）をやり取りして動くシステムです。

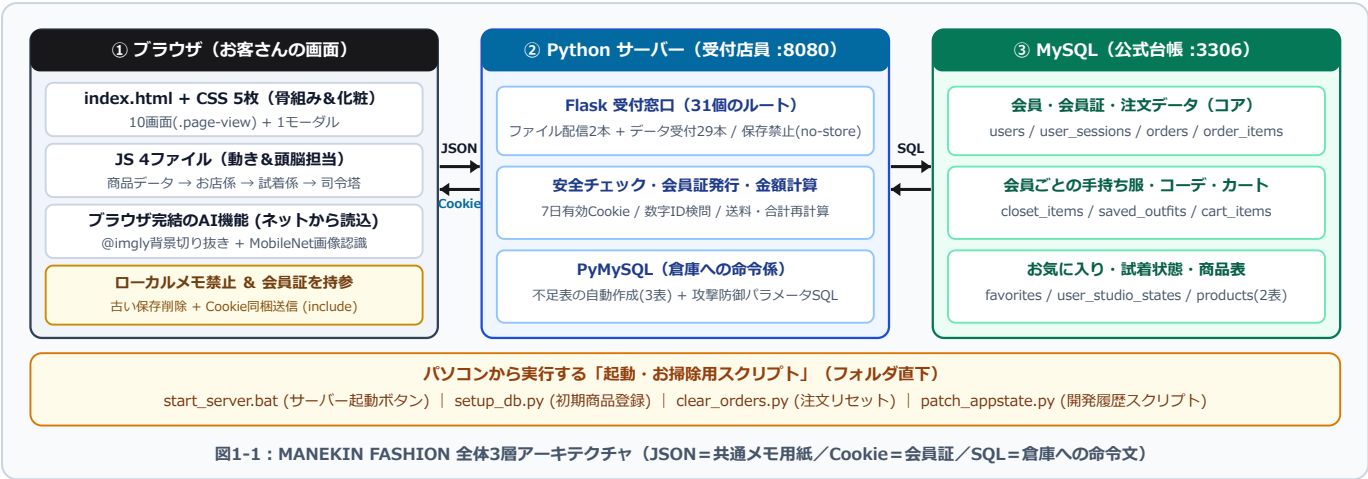


図1-1：MANEKIN FASHION 全体3層アーキテクチャ（JSON＝共通メモ用紙／Cookie＝会員証／SQL＝倉庫への命令文）

全15ソースファイルの実測構成一覧（この15個ですべて動いています）

プログラムが長くなりすぎて読めなくならないよう、デザイン担当（CSS）は5個、動き担当（JS）は4個に役割分担しています。

ファイルパス	担当	行数	コメント	Bytes	初心者向けの役割説明・中に書かれていること
`index.html`	骨組み(HTML)	2,624	989	247,160	全10画面分の文章やボタン・1つの登録窓・外部AIとJSの読込指示
`css/style.css`	化粧の入口	19	9	1,216	下の4つのCSSファイルを「この順番で全部読み込め」とまとめる目次役
`css/base.css`	基本デザイン	842	370	46,902	ブランド共通の色（黒など）・文字サイズ・固定ヘッダー・共通ボタン
`css/shop.css`	EC画面化粧	3,310	1,400	175,166	トップ大画像(HERO)・商品カード・レジ・ログイン・マイページの飾り
`css/studio.css`	試着室化粧	4,222	1,809	230,564	3列の試着スタジオ・5層の重なり座標・位置スライダー・AI判定結果枠
`css/responsive.css`	端末別調整	2,017	809	113,418	PC/タブレット/スマホの横幅（`1100/992/768/360px`）に合わせた画面整形
`js/products-data.js`	商品カタログ	761	373	55,006	公式アイテム全24着（男性12点・女性12点）の名前・価格・写真一覧
`js/app.js`	全体司令塔	727	327	45,986	画面の作業メモ `AppState`・起動時のログイン確認待ち・画面切替係
`js/shop.js`	お買い物係	3,005	1,207	209,968	会員証でのログイン維持・アカウント切替・カート・ポイント計算・注文確定
`js/studio.js`	試着室係	4,200	1,678	288,039	5枚重ね表示・ドラッグ装着・サイズ微調整・AI背景透過＆服判定・DB保存
`app.py`	店員(Python)	2,375	1,063	179,016	裏方サーバー本体。全39個の部品関数・31個の受付窓口・Cookie認証・DB操作
`setup_db.py`	初期データ	160	68	9,206	最初の商品画像パスと色バリエーション19件をMySQL台帳へ書き込む
`clear_orders.py`	注文消去	69	32	4,249	テスト注文を綺麗にしたいとき「明細→本体」の順で安全に消去する
`patch_appstate.py`	開発補助	29	15	2,037	制作途中で1行だけコードを足すために使った履歴スクリプト
`start_server.bat`	起動スイッチ	43	20	2,727	文字化け防止（UTF-8）を設定してから8080番で `app.py` を立ち上げる
合計（15ファイル）	**Full-Stack**	**24,403**	**10,169**	**1,610,660**	**合計約 1.61 MB（コメント率 41.7%・実ファイル最新実測値）**

第2章 1枚のHTMLで画面を切り替える仕組み（SPA）・CSS・商品データ

1. SPA（シングルページアプリケーション）：別のページへ飛ばない画面切り替え

ふつうのWebサイトはボタンを押すたびに別のHTMLファイルを読み直しますが、本サイトは **「1冊の本（`index.html`）の中に最初から10ページ分の内容（`.page-view`）を全部入れておく」** 作り方をしています。普段はCSS（`display: none`）で隠しておき、URL末尾の「`#○○`（しおり）」が変わった瞬間だけ、見せたいページに `.active` の付箋を貼り替えて表示します。画面を読み直さないのでサクサク動きます。

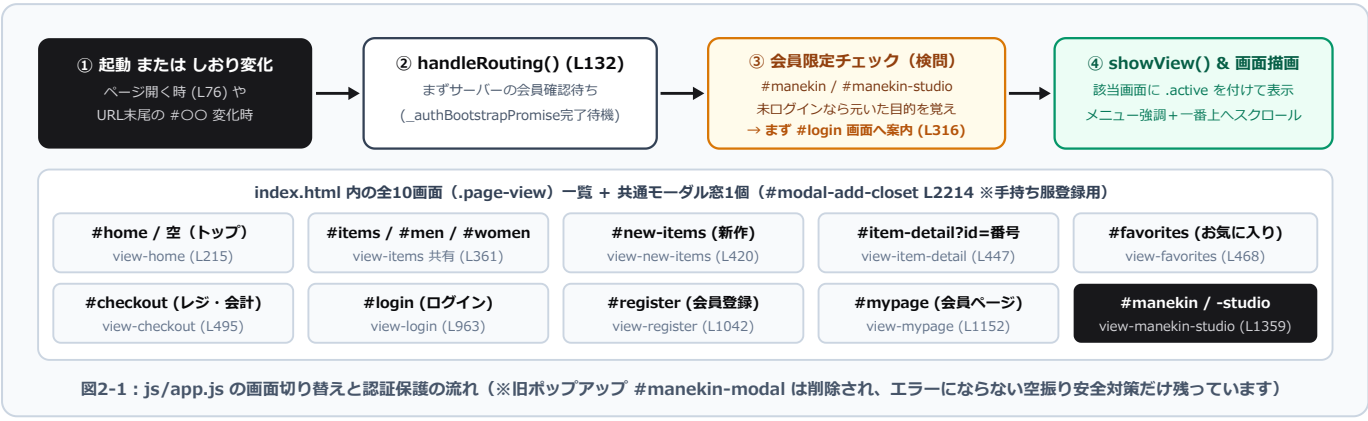


図2-1：js/app.js の画面切り替えと認証保護の流れ（※旧ポップアップ #manekin-modal は削除され、エラーにならない空振り安全対策だけ残っています）

2. JavaScript の読み込み順番（お料理と同じ下ごしらえ順・L2586-2619）

1. `<script type="module">`：先にネット上からAI背景透過プログラム（`@imgly`）を取り寄せて `window.imglyRemoveBackground` に置く
2. TF.js v4.10.0 + MobileNet v2.1.0：画像を判定するAI頭脳を読み込む
3. `js/products-data.js`：まず「全24商品リスト（`SAMPLE_PRODUCTS`）」を用意
4. `js/shop.js`：会員証チェック・カート・注文・マイページの道具を用意
5. `js/studio.js`：試着マネキン・AI切り抜き・クローゼットの道具を用意
6. `js/app.js`：全部品が揃った最後に作業メモ `AppState` を作り起動する

3. CSS（見た目担当）の5ファイル分割と役割 (css/style.css)

`index.html`（L29）は入口の `css/style.css` だけを1本読み込み、中身の `@import`（合体命令）が上から順番に見た目を当てます。

- `css/base.css`：共通ルール（黒 `#111111`、ヘッダー高さ `72px`、最大幅 `1200px` 等）
- `css/shop.css`：トップ写真枠（高さ `85vh`）・4列カード・レジ画面・マイページ
- `css/studio.css`：試着室3列レイアウト（左 `310px` / 中 `1fr` / 右 `340px`）・5層座標
- `css/responsive.css`：最後に読み込み、画面幅（`1100 / 992 / 768 / 360px`）別にスマホ最適化で上書き

4. 今開いている画面の「作業メモ帳」AppState (js/app.js L9-66) & 商品マスター 24着

`AppState` は、ページを開いているあいだ中プログラム同士で共有する**机の上のメモ帳（JavaScript変数）**です。「ログイン会員 `currentUser`」「ログイン後に自動ジャンプしたい戻り先 `pendingAuthTarget`（`'manekin'` または `'checkout'`）」「カート `cart`」「お気に入り `favorites`」「マネキンに着せている5部位 `currentOutfit`」「位置微調整値 `fittingOffsets`」を記憶します（ページを閉じても消えない保存はすべて第4・6章の MySQL が担当）。商品一覧 `SAMPLE_PRODUCTS` には `id: 1〜12`（MEN 12着）と `13〜24`（WOMEN 12着）の計 **24着** が入り、MySQL `/api/products` から色別の画像（`colorImages{}`）も合体させて使えます。

第3章 お買い物計算・ポイント精算・失敗ゼロの安全レジ処理

レジ画面の計算（`js/shop.js L1159, L1490`）と注文受付（`app.py POST /api/orders L303`）は、「ポイント計算」「金額の改ざん防止」「購入商品を自動で試着室へ届ける連動」を、途中で失敗したらすべて帳消しにできる安全な1セット処理（トランザクション）で実行します。

ポイント制度（お得なルール）

- **入会特典：**新規登録・デモ体験会員ともに最初 `1,000 pt` プレゼント
- **割引価値：** `1 pt = 1 円` 引き
- **還元計算：**ポイント利用後の支払額 `netTotal` の **1%**（1円以上なら最低 `1 pt` 保証）

送料と請求額・消費税の計算

- **商品小計：**全商品の「単価 × 個数」の合計
- **配送料：**小計 5,000円以上 **無料** / 5,000円未満 **550円**
- **ポイント上限：** `min(保有pt, 小計 + 送料)`
- **支払総額：** `netTotal = max(0, 合計 - 使用pt)`
- **内税表示（目安）：** `Math.round(netTotal * 10 / 110)`

買った服を自動で試着室へ

- 注文と同時に MySQL の `closet_items` 表へ「購入品（`is_purchased=1`）」として自動登録
- **整理コード規則：** `purchased-{商品ID}-{注文番号}-{連番}`
- **過去注文の後追い取込：** `importOrderToCloset(idx)`（L1330）でも同じルールで取込可能

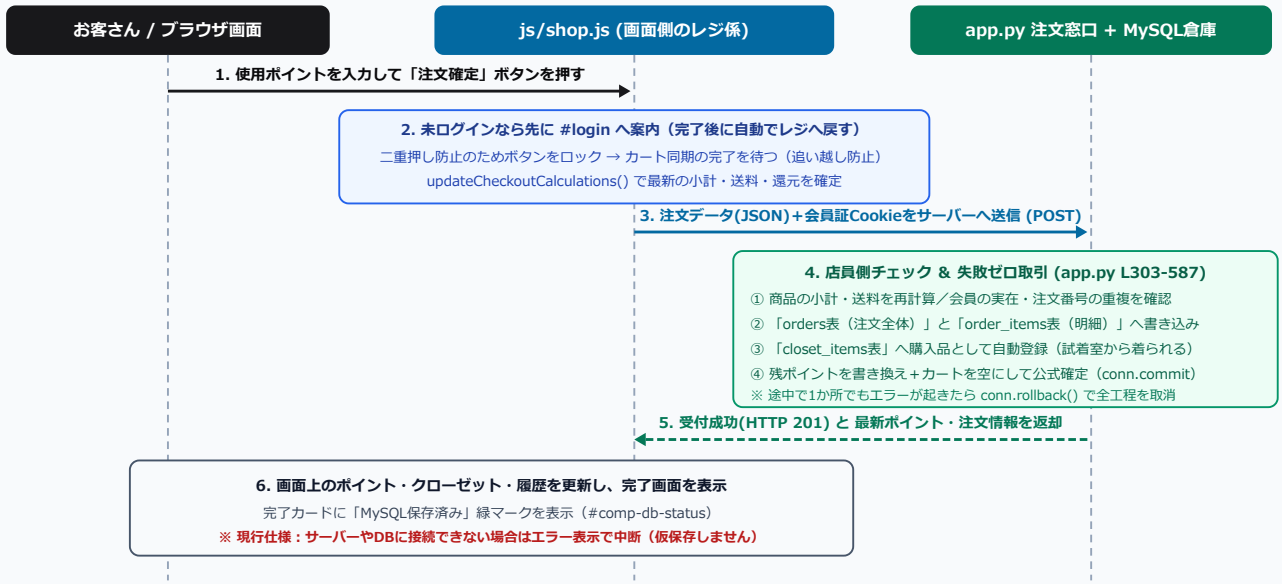


図3-1：購入確定ボタンから MySQL 確定・ポイント清算・試着室自動登録までのシーケンス（時間順の流れ図）

初心者メモ：なぜサーバー店員側（app.py）でも小計・送料を計算し直しているの？

ブラウザ上で動く JavaScript の金額は、ブラウザの開発者ツールを使えばお客さん側で書き換えることも技術的にできてしまいます（たとえば「送料0円・合計1円」に改ざんするなど）。それを防ぐため、`create_order()`（`app.py L339-357`）では届いた商品の単価と個数から小計や5,000円送料ルールをサーバー側で必ず計算し直し、正しい請求額を台帳（MySQL）に記録します。また「4つ全部成功なら `commit`（確定）、1つでも失敗なら `rollback`（最初から帳消し）」とすることで、「お金・ポイントだけ引かれて注文が残らない」事故を防いでいます。

第4章 別人のデータが混ざらない安全設計（Cookie会員証 × 整理番号ガード）

同じパソコンでAさんがログアウトしてBさんがログインしたとき、「前の人（Aさん）のカートや注文履歴が残って見えたら大問題」です。現行コードではブラウザ側の保存箱（`localStorage`）を全廃し、**①封印付き会員証（HttpOnly Cookie）での本人確認、②前の人の記憶を消すキャッシュ禁止設定、③通信の順番追い越しを防ぐ整理番号**の三重の防御を入れています。



図4-1：MySQL一本化・Cookie会員証・前ユーザーのデータ残存を防ぐ3重バリア

よくある疑問：難しい名前「世代ガード」「デバウンス」「フラッシュ」ってどういう意味？

- **① 世代ガード（`_MutationSeq` ＝ 通信の整理番号札）：** 「ログイン直後の履歴取得」のように時間がかかる通信の途中で、ユーザーが素早く「手持ち服追加」や「別アカウント切替」をすると、最初に送った古い問い合わせの返事が後から届いて新しい画面を上書きする事故（レースコンディション）が起きます。そこで通信前に番号札（`1番→2番→3番...`）を取り、返事が届いたとき「今の最新番号や今の会員IDと一致していなければその返事を捨てる」仕組みが世代ガードです。カート同期も `_cartSyncPromise` で1列に順番待ちさせます。
- **② 400ms デバウンス（連打まとめ待ち）& フラッシュ（待たずに即確定）：** 試着室で位置スライダーを右にズルズル動かすと1秒間に何十回も変更が起きます。毎回 MySQL に保存するとサーバーがパンクするため、`syncStudioStateToBackend()`（`js/studio.js L3980`、実行タイマーは `400ms` at L4032 ※コードコメントには220ms表記あり）は「操作の手が止まって0.4秒経ったら最後の状態だけ1回送る」工夫（デバウンス）をしています。逆にログアウトする瞬間だけは待てないので、`flushPendingStudioStateSync()`（L3953）が0.4秒タイマーを中止して即保存を済ませてから退室します。

第5章 MANEKIN AI STUDIO（5枚重ねマネキン・AI切り抜き・服判定）

試着室（`js/studio.js` 4,200行・`css/studio.css` 4,222行）は、透明背景の洋服画像をトランプのように5層に重ねて着せ替えでき、さらに手持ち服の写真を撮るだけで「背景を自動で切り抜き、服以外の写真（犬や風景など）をブロックするAI」を搭載しています。

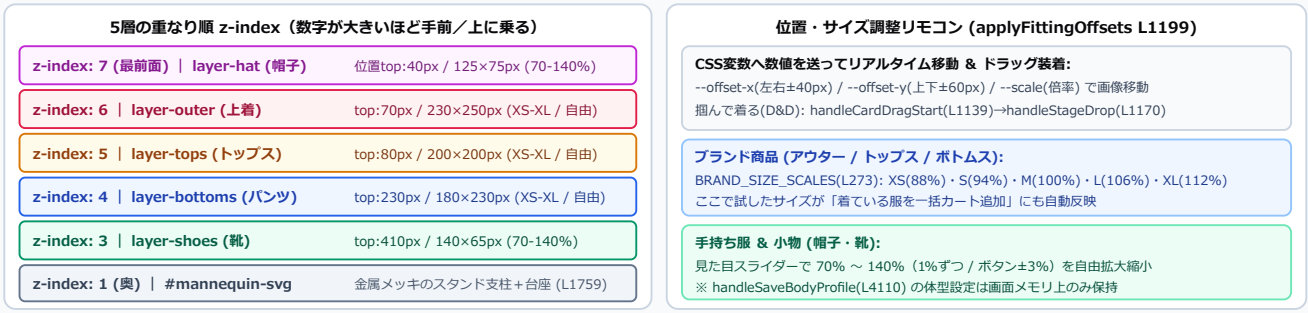


図5-1：マネキン5層の重ね順（z-index）とドラッグ着せ替え・サイズスライダー連動機構

どんな写真でも登録できてしまう問題を防ぐ「AI背景切り抜き & 2段階チェック」

手持ち服の登録窓（`#modal-add-closet`）では、写真を選ぶとまずサーバーへ送らず**自分のブラウザの中だけで**背景を透明化し、続いて形と中身を検査する関数 `analyzeClothingItemShape`（`js/studio.js L2773-3282`）が合格（`isValid !== false`）を出した写真だけ保存できるようにしています。

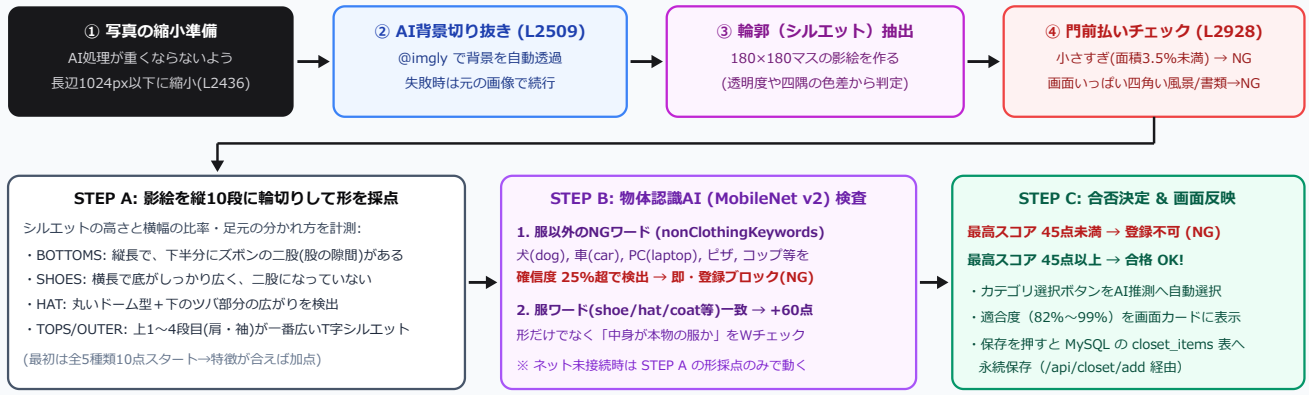


図5-2：ブラウザ内 AI 切り抜き（外部に写真を送らず安全）と5カテゴリ形状・物体認識の二重チェック

第6章 データベース（MySQL manekin_db）全11個の「保存表」ガイド

MySQL（`127.0.0.1:3306 / manekin_db`）の中には**11種類の表（テーブル）**があり、会員データをバラバラにせず「会員番号（`user_id`）」のヒモでしっかり結んでいます。表の見方のコツ： `PK` =行ごとの出席番号（絶対重複なし）／ `UNI` =同じ値の二重登録を禁止する鍵／ `FK` =別の親テーブルと結ぶヒモ（`CASCADE` =親が退会したら子のデータも連動して自動消去）。

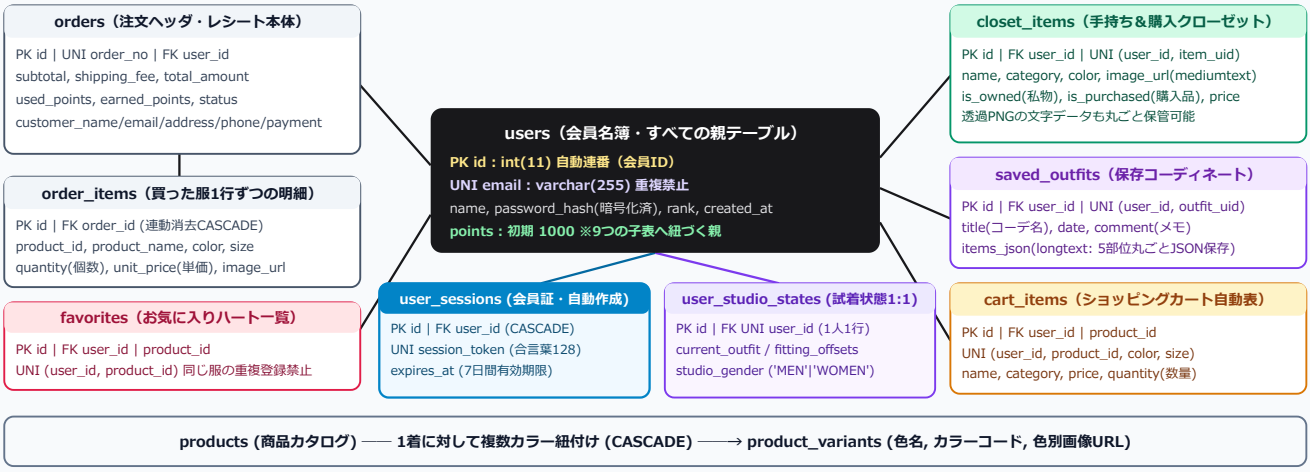


図6-1：MySQL (manekin_db) 全11テーブルのつながり図（ER図）— 退会時にゴミが残らない CASCADE 連動

全11テーブル詳細定義一覧（Part 1：会員・認証・商品・注文・お気に入り）

表の名前	番号・重複禁止・ヒモ	入っている項目（カラム）とデータ型・役割
`users`	PK `id` UNI `email`	`id int(11)`, 名前 `name`, メール `email`, 暗号化パスワード `password_hash`, 会員ランク `rank`, 作成日時 `created_at`, 保有ポイント `points`（初期値 1000）
`user_sessions` ★新	PK `id` FK `user_id` UNI `session_token`	`session_token varchar(128)`, `user_id int(11)`, 有効期限 `expires_at datetime`（Cookie会員証の本物確認台帳・7日間有効）
`products`	PK `id`	性別 `gender`, 日本語/英語名 `name/name_en`, 部位 `category`, 価格 `price`, 説明 `description`, 素材 `material`, 型番 `sku`, 新作/注目フラグ, 画像パス
`product_variants`	PK `id` FK `product_id`	`product_id int(11)`, 色名 `color`, 見本丸色 `color_code varchar(20)`, 色別画像URL `image_url`（色違い写真用。`setup_db.py` で19件投入）
`orders`	PK `id` UNI `order_no` FK `user_id`	注文番号 `order_no`, 配送先(`customer_name/email/postal_code/address/phone`), 支払・配達時間, `subtotal`, `shipping_fee`, `total_amount`, `used/earned_points`, `status`
`order_items`	PK `id` FK `order_id`	どの注文か `order_id`, 商品番号・名前・色・サイズ(`varchar(10)`)・数量 `quantity`・単価 `unit_price`・画像 `image_url`
`favorites`	PK `id` FK `user_id` UNI `(uid, pid)`	`user_id`, `product_id`, 登録日時 `created_at`（同じ人が同じ商品を2回「お気に入り」にしてもエラーにならず1つだけ保持）

全11テーブル詳細定義一覧（Part 2：試着室・コーデ・カート・自動状態保存）

表の名前	番号・重複禁止・ヒモ	入っている項目（カラム）とデータ型・役割
closet_items	PK id FK user_id UNI (user_id, item_uid)	固有文字ID item_uid, 服の名前・カテゴリ・色・季節, image_url mediumtext（透過画像の文字データ対応）, 私物フラグ is_owned, 購入フラグ is_purchased, 価格・購入日
saved_outfits	PK id FK user_id UNI (user_id, outfit_uid)	固有ID outfit_uid, コーデ名 title, 保存日 date, AI感想コメント comment, 5部位すべての着用服まとめ items_json longtext
cart_items	PK id FK user_id UNI uk_user_cart_item	商品ID product_id, 名前・カテゴリ・色・サイズ(varchar(20) DEFAULT 'M')・価格・個数 quantity・画像。一意キー(会員,商品,色,サイズ)で重複統合
user_studio_states	PK id FK user_id UNI uk_studio_state_user	1人1行で試着中の状態を保存：current_outfit_json longtext（着用中5部位）、fitting_offsets_json longtext（位置ズレ/サイズ）、男女モデル studio_gender

データベースの「設計の工夫」（なぜこの型やルールを選んだの？）

1.「箱の大きさ」の使い分け

名前やメールは短い箱 varchar(255) にして検索を高速化。一方、AI透過後の画像データは文字に直すと数十〜数百KBと大きいため最大16MB入る特大箱 mediumtext（ closet_items.image_url ）を、5部位のまとめJSONには longtext を割り当て画像欠損を防ぎます。

2. 重複しても壊れない仕組み

「同じ人の同じ服」が二重登録されないよう複合一意キー（ UNIQUE ）を張り、さらに保存SQLに ON DUPLICATE KEY UPDATE を書くことで「初なら新規追加・すでに在れば最新値で安全に上書き（冪等＝何回送っても壊れない）」を実現しています。

3. 起動時の自動点検（app.py L111）

cart_items・user_studio_states・user_sessions の3表は、app.py 起動時に ensure_account_tables()（L111）が CREATE TABLE IF NOT EXISTS（「まだ無ければ表を作れ」）を自動実行して作り忘れを防ぎます。

初心者メモ：なぜ「orders」と「order_items」の2つの表に分けるの？（第1正規化）

1回の買い物で服を3着買うとき、orders の1行に無理やり「商品1, 商品2, 商品3...」と横長の列を作ると4着以上買ったとき破綻します。そこで「お会計全体（誰が・いつ・何pt使って・いくら払ったか）」を親の orders 表に1行だけ書き、「買った服ごとの明細（商品名・色・サイズ・単価・個数）」を子の order_items 表に3行並べることで、何点まとめて買っても綺麗に管理できるようになります。

よく出るデータベースの専門用語・かんたん早見表

int(11) = 整数（年齢・価格・番号）／ varchar(N) = 最大N文字までの短い文字列 ／ text / mediumtext / longtext = 長い文章やデータ用の箱 ／ datetime = 年月日時分秒 ／ DEFAULT ... = 値が空なら自動で入れる初期値 ／ DDL = データではなく「表の設計図そのもの（ CREATE TABLE 等）」を作る命令。

第7章 裏方サーバー（app.py）の全31個の「受付窓口（ルート）」完全ガイド

`app.py`（**2,375行**）には全 **39個** のPython関数（`def`＝仕事ごとの道具箱：窓口担当31個＋裏方ヘルパー8個）と **31個の受付窓口（API 29本 + ファイル配信 2本）** が用意されています。どの窓口も `parse_user_id()`（L94）と `WHERE user_id = %s` で「他人のデータを見せない」チェックを通ります。

種類	受付窓口のURLパス	担当Python関数 / 行	やっている仕事と安全チェックのポイント
GET	/api/products	get_all_products (L191)	MySQLから全商品と色違い画像（`product_variants`）を合体して返す
GET	/api/products/<int:product_id>	get_product_detail (L238)	指定した商品番号1着の詳細・色別画像・サイズを返す（無ければ404）
GET	/api/health	health_check (L285)	サーバーとMySQL倉庫がちゃんと繋がっているかの健康診断窓口
POST	/api/orders	create_order (L303)	総額・送料を再計算し、注文登録・試着室自動追加・pt更新・カート空化を一括実行
GET	/api/orders	get_orders (L592)	`?user_id=` で指定された本人の注文履歴＋明細だけを新しい順に取り出す
DEL	/api/orders	clear_orders (L677)	指定した本人の注文明細→注文本体だけを消去する（他人の分は保護）
POST	/api/auth/register	register_user (L779)	メール重複(409)確認後、初期1,000ptで会員登録(201)し会員証Cookieを発行
POST	/api/auth/login	login_user (L864)	メール＆暗号パスワードを照合し会員情報＋7日有効Cookieを発行（不一致なら401）
POST	/api/auth/demo	demo_login_user (L929)	体験アカウント（`demo@manekin.com`）を探し、無ければ自動作成してCookie発行
GET	/api/auth/me ★新	get_current_auth_user (L1006)	手持ちCookieと `user_sessions` を照合し本人を返す（再読込時のログイン復元用）
POST	/api/auth/logout ★新	logout_user (L1072)	`user_sessions` の会員証データを削除し、ブラウザのCookieも即時無効化
POST	/api/user/sync	sync_user_data (L1108)	会員の保有ポイント値を MySQL の `users.points` へ同期更新する
GET	/api/favorites	get_favorites (L1158)	指定ユーザーがお気に入り登録している商品IDのリストを取得
POST	/api/favorites/toggle	toggle_favorite (L1197)	お気に入りに既にあれば削除（解除）、無ければ追加（ON/OFF切替）
POST	/api/favorites/sync	sync_favorites (L1255)	渡されたお気に入りID配列を `INSERT IGNORE` で重複なし一括登録
GET	/api/outfits/<int:user_id>	get_user_outfits (L1313)	`saved_outfits` 表から本人の保存コーデ一覧を取得し5部位JSONを復元
POST	/api/outfits/save	save_user_outfit (L1368)	コーデとAI診断コメントを保存/上書き更新して 201（作成完了）を返す
DEL	/api/outfits/delete/<outfit_uid>	delete_user_outfit (L1435)	コーデIDと会員番号の両方が一致する本人の保存コーデだけを削除

初心者メモ：窓口表に出てくる「GET・POST・DEL」や「<int:user_id>」の読み方

① **3つの通信合図（HTTPメソッド）：** `GET` は「データを見せて・ちょうだい（読み取り）」、`POST` は「新しいデータや計算をお願い（送信・登録）」、`DEL`（`DELETE`）は「このデータを消して（削除）」という約束事です。

② **パスの中の <int:user_id> とは：** 「ここにはユーザーの番号（`int`＝整数、例：`/api/outfits/5`）」が入る穴埋め枠です。★印の `/api/auth/me` と `/api/auth/logout` は、Cookie認証でリロード維持・完全ログアウトを行うため新しく備わった重要ルートです。

全31ルート完全ガイド（後半：3D試着・クローゼット・カート・スタジオ・静的配信）

種類	受付窓口のURLパス	担当Python関数 / 行	やっている仕事と安全チェックのポイント
GET	/api/manekin/avatar/<user_id>	get_manekin_avatar (L1505)	将来の3Dマネキン拡張用プロフィール（身長172cm等）の初期値を返す窓口
POST	/api/manekin/try-on	manekin_virtual_try_on (L1522)	仮想試着プレビュー＆フィット診断（ OPTIMAL_FIT ）の簡易スタブ応答
GET	/api/closet/<user_id>	get_user_closet (L1560)	closet_items 表から本人の手持ち服・購入品を新しい順(id DESC)で取得
POST	/api/closet/add	add_to_closet (L1614)	カメラやファイルから追加した透過済み手持ち服等を MySQL へ保存(201)
DEL	/api/closet/delete/<item_uid>	delete_closet_item (L1738)	アイテムIDと user_id の両方が一致した本人の私物アイテムだけを削除
POST	/api/ai-coordinate/recommend	recommend_coordinates (L1785)	天気・気温・シーンを受け取り、おすすめコーデと信頼度 0.94 を返却
GET	/api/cart/<int:user_id>	get_user_cart (L1839)	cart_items 表から指定アカウントのカート商品を登録順に取り出す
POST	/api/cart/sync	sync_user_cart (L1911)	同じ(商品・色・サイズ)を合算し「旧行全削除→新行全挿入」で綺麗に同期
DEL	/api/cart/<int:user_id>	clear_user_cart (L2058)	指定した会員本人の cart_items だけを一括で空にする
GET	/api/studio-state/<int:user_id>	get_user_studio_state (L2116)	user_studio_states から前回の着用5部位・位置調整・性別を取得
POST	/api/studio-state/save	save_user_studio_state (L2234)	今の試着状態・微調整値・性別を ON DUPLICATE KEY UPDATE で自動保存
GET	/	serve_index (L2317)	ブラウザで http://localhost:8080/ を開いたとき index.html を渡す
GET	/<path:path>	serve_static (L2324)	css/* ・ js/* ・ assets/* ・ images/* の実ファイルをそのまま届ける

裏方サーバーの「安全守則」を初心者向けにかみ砕くと？

悪い入力からデータベースを守る3つの盾

- **SQLインジェクション対策（%s 埋め込み）：**入力欄に悪い命令文を紛れ込ませてデータを壊す攻撃を防ぐため、文字を直接つなげず必ず安全な枠（ %s プレースホルダ）へ渡しています。
- **フェイルクローズ（怪しいIDは即お断り）：** parse_user_id() （L94）が文字や 0 やマイナス値を受け取ったら一律「無効」とみなし、絶対に他人のデータを出しません。
- **ブラウザへの記憶禁止（no-store）：** disable_api_cache() （L47）が全 API の返事へ「ブラウザは画面を覚え込むな」という指令（ Cache-Control: no-store ）を付けます。

会員証・お返事コード（HTTPステータス）・CORS

- **封印付き会員証（HttpOnly Cookie）：** manekin_session （7日間有効）には httponly=True を付け、悪意あるJSプログラムから盗み読まれるのを防ぎます。
- **結果コード（番号の意味）：**200台＝成功（ 200 完了 / 201 新規作成）、400台＝入力や権限不備（ 400 不正 / 401 パスワード違い / 404 未存在 / 409 重複）、 500 ＝サーバー・DB側の不調。
- **CORS（会員証つきの通信許可）：**ポート番号が違う開発環境からでも Cookie 付きで通信できるよう supports_credentials=True （L37）を設定し、 PORT=8080 で待ち受けます。

第8章 起動＆お掃除スクリプト ＆ 初心者向けキーワード辞書

1. フォルダ直下にある4つの運用・補助スクリプトの使い方

ファイル名	動かし方・行数	いつ、何のために使うのか
`start_server.bat`	ダブルクリック / 実行 (43行)	黒い画面の文字コードを UTF-8（`chcp 65001`：日本語文字化け防止）にしてから `python app.py` を起動します。必要ライブラリが足りない時は `pip install ...` コマンドを案内して止まります。
`setup_db.py`	`python setup_db.py` (160行)	MySQL に `manekin_db` データベースが無ければ作成し、初期7商品の画像パス更新・色別画像19件の投入・古いテスト商品(ID:8)の削除を行います。
`clear_orders.py`	`python clear_orders.py` (69行)	テストで作った注文履歴を全部消してリセットしたい時に使います。外部キーの親子エラーを避けるため、必ず「子 `order_items` → 親 `orders` 」の順番で削除します。
`patch_appstate.py`	開発時の補助パッチ (29行)	開発途中で `js/app.js` に1行だけコードを自動挿入するため使った履歴スクリプトです（現行コードに実行すると「すでに対象箇所が見つかりません」と出て安全に終了します）。

2. はじめてコードを読む人のための「横文字かみ砕き」用語集

SPA（シングルページアプリケーション）

たとえ：1冊の飛び出す絵本 ページを移動し直さず、1つの `index.html` の中で JavaScript が見せたい画面（`#home` 〜 `#manekin-studio`）だけを素早く入れ替える仕組みです。

REST API & JSON（fetch通信）

たとえ：注文用紙でやり取り 裏側からサーバー窓口（`/api/...`）へ連絡し、世界共通の文字形式「JSON（キー:値）」でやり取りして、画面を閉じずにDB保存できる仕組みです。

HttpOnly Cookie & キャッシュ禁止 (no-store)

たとえ：封印シール付き会員証 プログラムから中身を読めない Cookie（`manekin_session`）で会員を識別し、`no-store` でブラウザに古い別人の返事を記憶させません。

トランザクション (commit / rollback)

たとえ：全勝か全キャンセル 「注文書・明細・ポイント引き・カート空化」など複数作業を「全部成功したら `commit`（確定）」「1つでも失敗したら `rollback`（全帳消し）」にする機能です。

外部キー (FK) & ON DELETE CASCADE

たとえ：親とつながる整理ヒモ `orders.user_id` のように会員テーブルと結ぶ番号列。`CASCADE` 設定により、会員が削除されると関連データもゴミを残さず連動消去されます。

Upsert (ON DUPLICATE KEY UPDATE)

たとえ：無ければ作成・あれば上書き MySQL に同じ `UNIQUE` キーがまだ無ければ新規追加（INSERT）、すでに存在したらエラーにせず最新データへ上書き（UPDATE）する便利な書き方です。

デバウンス (Debounce) & 世代ガード

たとえ：一呼吸待つ & 整理番号 連打中に通信を節約するため「止まって0.4秒（400ms）待つ」のがデバウンス。遅れて届いた古い返事で上書きされないよう番号札で弾くのが世代ガード（`_MutationSeq`）です。

WASM / ブラウザ完結型 AI 推論

たとえ：PC内で動く高速AIアプリ `@imgly`（背景切り抜き）や `MobileNet v2`（物体認識）は、写真を外部サーバーへ送らず自分のブラウザ内だけで計算するため安心・高速に動きます。

初心者向け・コード読み進め方 & 検証ステータス（HEAD 3eb78a6 静的精査済み）

おすすめの読み順：① まず `index.html` の各 `.page-view` と `js/app.js` で「画面の全体地図」をつかむ → ② 見たい機能の画面（買い物なら `js/shop.js`、試着なら `js/studio.js`）を開き、そこから呼ばれる `app.py` の窓口を第7章の行番号で開く、の順がおすすめです。

※ 本書（全15ファイル・**24,403行**・コメント **10,169行 / 41.7%**・31ルート・11テーブル）の数値と仕様は現行リポジトリ（HEAD `3eb78a6`）の静的ソース精査に基づいており、動的確認は Flask + XAMPP MySQL 起動環境で行ってください。